Jones Automation Exercise

Part B: Billing UI Mock-up Analysis, Test Cases & Product Solution

---

Part B – Q1: Billing UI Mock-up Analysis

1. Security & Compliance

a. Definite Security Issues Found (Observed directly from the UI mockup)

- **Missing CVV / CVC Field (Critical):** There is no field to enter the 3-4 digit card verification code (CVV/CVC). Almost all online credit card transactions require this code; without it, physical possession of the card cannot be validated, exposing the system to Card-Not-Present (CNP) fraud, and contrary to card-network and payment-processor requirements.
- **Lack of Security Indicators:** There is no indication that the page is secure (e.g., HTTPS protocol indication, browser lock icon, PCI-DSS compliance badge, or SSL encryption logo), which undermines user trust when entering sensitive financial details.

b. Potential Security Issues (Require further verification / Not observable in static mockup)

- **Input Restrictions & Client-Side Validation:** The mock-up does not show any input restrictions or client-side validation (Luhn check, allowed characters, field length limits, or BIN alignment between Card Type and Card Number). This should be verified on the real form.
- **Card Number Exposure, Encryption, and Tokenization:** The Card Number field displays full digits without masking options during or after input. Verification is required to ensure end-to-end encryption/tokenization and to guarantee that raw card numbers are never stored in system logs, URL parameters, or browser cache.
- **Injection Attacks (XSS / SQL Injection) in Free-Text Fields:** Open text fields (e.g., Street Address, City, Name) require strict client-side and server-side input sanitization and escaping to prevent malicious script or query execution.
- **Lack of Multi-Factor Authentication (2FA / 3D Secure):** It is unknown whether the 3D Secure protocol (e.g., SMS OTP verification from the issuing bank) is triggered upon submission to authorize the transaction.
- **Lack of Rate Limiting / CAPTCHA Protection:** Exposure to automated carding attacks (scripted testing of stolen credit card numbers) in the absence of CAPTCHA challenges or IP-based rate limiting.
- **Parameter Tampering on Payment Amount:** The Payment Amount is displayed as static text (30.00). It must be verified that this value is securely calculated and enforced on the server side; if passed within the client payload, an attacker could manipulate the request to pay 0.01 instead of 30.00.

2. Usability & UX

a. Definite Usability Issues Found (Observed directly from the UI mockup)

- **Missing Country Field vs. Mandatory State or Province Field:** For a global SaaS company, omitting a Country selection while marking State or Province as a mandatory field (*) creates a logical blocker that prevents international users (e.g., from Europe, Israel, etc.) from completing registration.
- **Rigid, Non-Localized Postal Code Format:** Postal code formats vary globally (numeric in the US/Israel vs. alphanumeric in the UK/Canada), but the form lacks dynamic format adaptation.
- **User-Burdening Instruction: "Card Number (No dashes or spaces)":** The system offloads data formatting responsibilities onto the user. A modern UI should allow users to enter spaces or dashes freely and automatically clean/format the input.
- **Lack of Error Validation Feedback:** There is no visual design indicating how input validation errors will be presented to the user (e.g., inline error helper text, red field highlighting).
- **Payment Amount Lacks Currency Symbol:** The payment amount is shown purely as a number (30.00) without specifying the currency ($, €, ₪). In a global context, this ambiguity creates confusion and risks transaction chargebacks.

b. Potential Usability Issues (Require further verification / Not observable in static mockup)

- **Session Timeout Handling:** If a user pauses while filling out the form (e.g., to fetch their credit card), it is unclear whether the session silently expires, resets the form, or provides a privacy-conscious timeout notification.
- **Lack of Actionable Visual Feedback:** Without an immediate loading indicator (e.g., spinner, button disabled state) upon clicking Continue, users may perceive the UI as unresponsive.

3. Performance & Reliability

a. Definite Performance & Architectural Issues (Observed directly from UI design)

- **Architectural Dependency on Server Validation:** The static layout does not indicate any immediate client-side feedback mechanisms, creating architectural dependency on server responses for all form submissions.

b. Potential Performance & Reliability Issues (Require further verification / Not observable in static mockup)

- **Client-Side Validation & Server Load:** The mock-up does not show whether input validation is performed on the client side prior to submission; unvalidated requests could cause unnecessary server load and network latency.
- **Response Delay on Continue and Risk of Double Charging:** If clicking Continue does not immediately disable the button (Disable on Click) or show a loading state, users may click multiple times due to network latency, triggering concurrent payment requests.
- **Lack of Double-Submit / Idempotency Protection:** It is unknown whether the backend utilizes Idempotency Keys to prevent processing duplicate transactions in the case of repeated button clicks or network retries.

Part B – Q2: Sample Test Cases For Functionality Check

TC-01: Successful Payment with Valid Details (Happy Path)

- **Priority:** High
- **Preconditions:** User is on the Account Information screen; Payment Amount displays 30.00.
- **Test Data:**
  - **Card Type:** VISA
  - **Card Number:** 4111111111111111 (standard test card number)
  - **Expiration:** A future month and year
  - **First Name / Last Name:** John / Smith
  - **Billing Address:** 1 Main St
  - **City:** New York
  - **State:** New York
  - **Postal Code:** 10001
- **Steps:**
  - **1.** Select "VISA" in Card Type.
  - **2.** Enter the card number without dashes or spaces.
  - **3.** Select a future expiration month and year.
  - **4.** Fill in First Name, Last Name, address, City, State and Postal Code.
  - **5.** Click "Continue".
- **Expected Result:** No validation errors appear, and the user moves to the next step (confirmation / payment processing) with the amount 30.00 unchanged.

TC-02: Expired Card is Rejected (Negative, Date Validation)

- **Priority:** High
- **Preconditions:** Same as TC-01. All other fields filled with valid data.
- **Test Data:** Expiration Month: a month that has already passed; Year: the current year.
- **Steps:**
  - **1.** Fill all required fields with valid data.
  - **2.** Select an expiration month earlier than the current month, with the current year.
  - **3.** Click "Continue".
- **Expected Result:** The payment is blocked. A clear error message appears next to the Expiration field (e.g., "Card has expired"), the field is highlighted, and the entered data in other fields is preserved.

TC-03: Required Fields Left Empty (Negative, Mandatory Fields)

- **Priority:** Medium
- **Preconditions:** User is on the Account Information screen, all fields empty.
- **Steps:**
  - **1.** Leave all fields empty.

    - **2.** Click "Continue".
- **Expected Result:** The form is not submitted. Every field marked with * (Card Type, Card Number, Expiration, First Name, Last Name, Street Address, City, State, Postal Code) shows a clear error, and focus moves to the first invalid field.

** Python examples for the test cases are included in the project ZIP file.*

Part B – Q3: Product Solution For The Most Severe Bug

Problem Definition (The Bug)

The payment form lacks a Card Verification Value (CVV/CVC) field. Key card details—such as card number, expiration date, and cardholder name—are printed directly on the physical card and are frequently exposed in data breaches. Without a CVV field, an attacker holding stolen card details can easily complete an unauthorized transaction.

Why it is the Most Severe Bug (Severity Analysis)

- **Security & Fraud Mitigation:** Omitting the CVV eliminates the primary mechanism for verifying that the payer physically possesses the card during a Card-Not-Present (CNP) transaction, leading to high rates of fraudulent transactions and chargebacks.
- **In-Depth Scope Analysis:** Assuming this form is the primary payment path, the defect affects every card transaction processed through it, regardless of card brand, browser or region.
- **Compliance & Industry Standards:** Major credit card networks (Visa, Mastercard) and payment processors mandate CVV verification for Card-Not-Present online transactions.

Proposed Product Solution

1. UI Changes

- Add a mandatory field named Security Code (CVV/CVC) next to the expiration date fields, marked with an asterisk (*) to match other required fields.
- Dynamically adjust input character limits based on card type: 3 digits for Visa/Mastercard, and 4 digits for American Express.
- Include a visual help tooltip (?) illustrating where the security code is located (back of card, or front for Amex).
- Enforce numeric input masking (digits only) and trigger a numeric keypad on mobile devices (inputmode="numeric").
- Set autocomplete="cc-csc" to support browser autofill and password manager integrations.

2. Validation

- **Client-Side:** Restrict input to digits only, enforce expected character length based on detected card type, and display clear inline error messaging for missing or malformed inputs.
- **Server-Side:** Re-validate all fields on the backend prior to sending authorization requests to the payment gateway (client-side checks serve UI/UX convenience only).

3. Security, Regulation & PCI-DSS Compliance

- **Strict SAD Non-Persistence:** Under PCI-DSS, Sensitive Authentication Data (CVV/CVC) must never be stored after authorization. Ensure the CVV is strictly excluded from application logs, error monitoring tools (e.g., Sentry, Datadog), analytics platforms, and database backups.

- **Transit Security & Headers:** Transmit CVV data strictly within the body of an encrypted HTTPS POST request (TLS 1.2/1.3). Never pass the CVV via URL parameters. Enforce strict cache-control headers (Cache-Control: no-store, no-cache, must-revalidate and Pragma: no-cache) to prevent caching in browser histories, proxies, or CDNs.
- **Client-Side UI Hardening:** Mask the input (type="password" or character obfuscation) and enforce a strict Content Security Policy (CSP) to prevent DOM-based XSS attacks from reading user input.

4. Backend & Fraud Controls

- Include the CVV in the authorization payload sent to the payment gateway, automatically treating a CVV mismatch response as a declined payment.
- Implement strict rate-limiting on failed attempts to prevent automated brute-force attacks (as 3-digit codes have only 1,000 possible combinations).

Alternatives Considered

Option	Pros	Cons
Add CVV Field Only (Recommended First Step)	Fast implementation, low risk, closes the primary security gap.	Does not stop 100% of advanced fraud scenarios.
Add CVV + 3-D Secure (SMS/App Verification)	Maximum security, shifts fraud liability to the issuing bank (Liability Shift).	Adds user friction; may slightly reduce checkout conversion rate.
Redirect to Hosted Payment Page	Minimal PCI compliance burden on internal infrastructure.	Complete loss of UI control and brand integration.

Recommendation: Implement an embedded CVV field on the form as Phase 1, followed by a risk-based evaluation of 3-D Secure for high-risk transactions in Phase 2.

Acceptance Criteria

- **1.** The CVV field is visible and required for all card transactions.
- **2.** Visa and Mastercard fields accept exactly 3 digits; American Express accepts exactly 4 digits. Letters and special characters are rejected.
- **3.** Submitting the form without a valid CVV is blocked on the client side with clear inline error messaging.
- **4.** The CVV is never exposed in URLs, application logs, database tables, or browser cache.
- **5.** An incorrect CVV returns a user-friendly transaction decline message.

Success Metrics

- Fraud-related chargeback rate decreases.
- Overall payment authorization success rate remains stable.
- Checkout abandonment rate does not increase post-release.